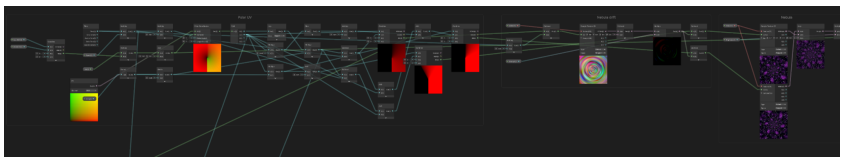
Technical Art — Shaders, Gyro Input, Player, & Enemies

Tubular Warpage · Unity URP · Samsung Galaxy Tab S9 FE / Android Phones / HTML-5

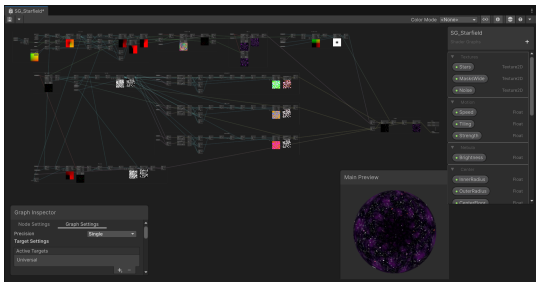
Where each effect lives and which exposed values may change to tune it. Node math is not covered (more details in the individual professional development reports) — all listed properties are on the Shader Graph blackboard and editable on the material in Editor / Play Mode, either to the right on the materials themselves, or, in the attached component script on the gameobject.

1 · Starfield — SG_Starfield

Tileable pixel-art star texture is warped into polar UV space and scrolled outward in two parallaxed layers with a time-multiplied noise distortion, so the stars stream past the player from center screen. RGB mask splits into three groups that are brightened and colored separately for control over 3 unique layers of stars, and the gyroscopes x and y vals also tilt the uvs in space further.



Polar UV, nebula drift and nebula groups

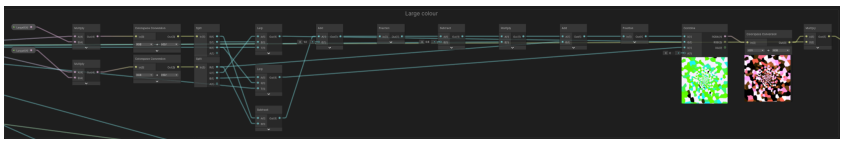


Full graph — exposed properties on the blackboard, right

- **Motion → Speed / Tiling / Strength** — outward drift, star density, tiling, noise distortion. High Strength smears the pixel art.
- **Center → InnerRadius / OuterRadius / CenterFloor** — size and falloff of the dark core that hides the polar-uv pinch point. **Nebula → Brightness** lifts the nebula layer only.
- **Textures → Stars / MasksWide / Noise** — swap art here. MasksWide packs the three star groups into R/G/B; keep that channel order or the per-group colours break. Shader would be extendable to work with other space/pixel-art textures.

2 · Star colour & twinkle

A black-and-white cell-noise mask, also in polar space, drifts outward and swaps each star group between two colours as it passes, as the cell noise's black and white cells pass over the RGB masked stars, this creates the twinkle. White = On, Black = Off



Large colour group — RGB → HSV, split, lerp, recombine

- **Blending happens in HSV, not RGB.** New colours must route through the existing Colorspace Conversion pair — multiplying in RGB blows out the midpoint hue (orange × pink reads green). This was a bug that took a long time to solve, ironically, the gyroscope input for the stars/background texture was something we thought would be hard, and that ended up being easiest to implement, whereas something simple - like twinkling stars back and forth between two colors - ended up being the biggest technical challenge to shader-creation and lookdev.

3 · Gyro input — Gyroscoper.cs

Raw angular velocity off the Input System device, integrated per axis each frame. Every angle reads as how far the tablet has turned about its own axis since neutral, and decays back toward neutral over time.

Why we chose to use the raw gyro, not the attitude sensor: attitude sensor fuses the gyroscope with the device's compass and accelerometer, so pitching or yawing the tablet bled into the roll reading — and roll is our steering wheel, so the ship drifted whenever a player simply tilted the device toward themselves (*which we only wanted to create x and y motion within space on

the stars/background shader). Reading the three raw gyro axes removed that cross-talk. Attitude worked for other teams' games; ours needed one clean axis for smooth input, steering, and parallax motions of the background.

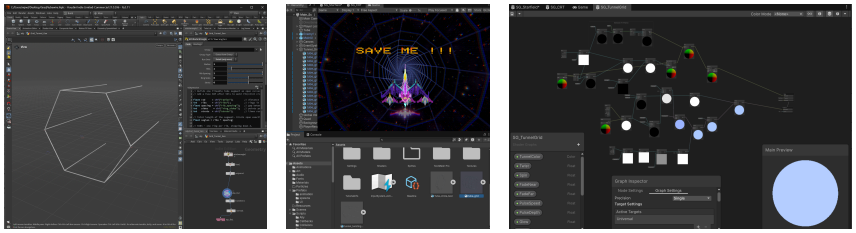
```
using Gyroscope = UnityEngine.InputSystem.Gyroscope;  // legacy Input.gyro shares the name
public float recenterRate = 0.2f;   // per sec; how fast a held tilt fades to neutral. 0 = never
public float holdRate     = 3f;     // per sec; the same fade while CENTER is held
public float deadband     = 0.5f;   // deg/sec; a still tablet must read as still
public float Roll  { get; private set; }   // about the screen normal - steering
public float Pitch { get; private set; }   // about the long edge
public float Lean  { get; private set; }   // about the short edge
public bool  Ready => Gyroscope.current != null;   // false in editor - callers fall back to keys

void Start() {   var g = Gyroscope.current;
    if (g != null && !g.enabled) InputSystem.EnableDevice(g);   }
void Update() {
    if (!Ready) return;
    Vector3 rate = Gyroscope.current.angularVelocity.ReadValue() * Mathf.Rad2Deg;
    rate = new Vector3(Dead(rate.x), Dead(rate.y), Dead(rate.z));  // below deadband is bias
    Vector3 turned = rate * Time.deltaTime;
    // Add this frame's turn, then fade toward zero: slowly on its own, fast while CENTER is held.
    float keep = Mathf.Clamp01(1f - (holding ? holdRate : recenterRate) * Time.deltaTime);
    Roll  = (Roll  + turned.z) * keep;
    Pitch = (Pitch + turned.x) * keep;
    Lean  = (Lean  + turned.y) * keep;
}
float Dead(float v) => Mathf.Abs(v) < deadband ? 0f : v;
```

- **deadband** — raise until a stationary tablet reads as still. Anything slower than this is sensor bias, not motion.
- **recenterRate / holdRate** — how fast a held tilt decays to neutral, and the faster decay while CENTER is held so neutral glides to wherever the tablet is being held.
- Set **recenterRate** to 0 while tuning to read the raw integrated angles.

4 · Tunnel — Houdini mesh & SG_TunnelGrid

The player orbits a fixed circle and never actually moves forward — a C# scroller drives identical tunnel segments toward the camera and recycles them to the back, which is what sells the motion. A VEX wrangle builds one tileable segment as curves that polywire thickens into a round grid. On the material, the vertex shader stage twists each segment around z as it travels and adds a roll fed from the gyro, while the fragment stage handles the emissive glow and the near/far fade in order to mask pop-in.

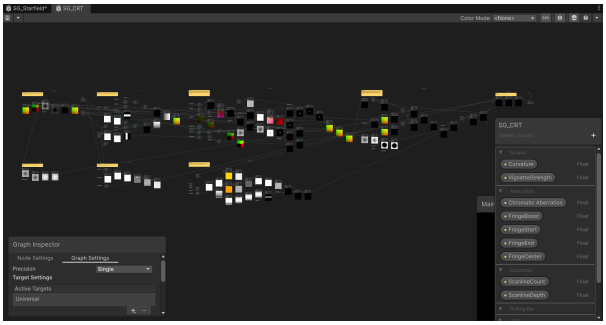


Houdini wrangle parameters · first tube-grid pass in engine · SG_TunnelGrid graph

- **Radius / Ribs / Rib Spacing / Ring Sides / Struts** — fewer sides and struts with wider spacing leaves more background visible. The first pass read as a spider web according to our art director; dropping sides down to a hex grid fixed it.
- **Segment Length** = Ribs × Rib Spacing. Keep it matched to the recycle distance on the tunnel scroller or the seams pop. Regenerate and re-export through **rop_fbx1** rather than editing the FBX. The updated mesh hot-reloads straight into Unity.
- **Twist / Spin** — constant z-rotation and the gyro-driven roll. Twist changes perceived speed as much as scroll speed does; Spin should read as a lean, not a barrel roll.
- **Glow / TunnelColor** — emissive strength and tint; compounds with bloom on the Global Volume.
- **FadeNear / FadeFar** — where segments fade in and out. Pull FadeFar in to hide the recycled pop-in
- **PulseSpeed / PulseDepth** drive the pulse & glow along the tube.

5 · CRT overlay — SG_CRT

A full-screen post-process that re-samples the rendered image through a curved-glass model: scanlines roll down it, the edges bow and fringe, and a centre warp hides the pinch point where the polar UVs converge. It sits above the game render, below the TV bezel sprite — moving it in the stack breaks that classic illusion.

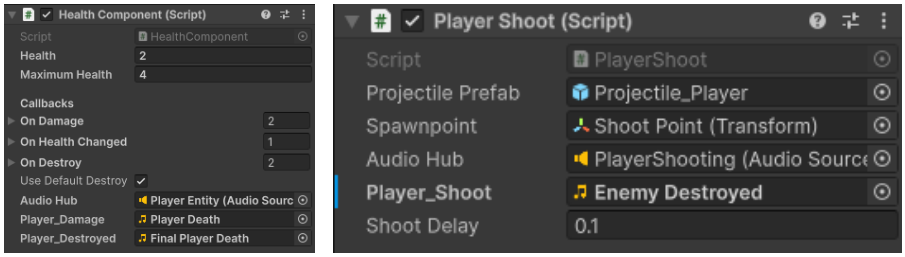


- **Screen → Curvature / VignetteStrength** — barrel distortion and edge darkening. (*makes screen looked curved slightly)
- **Aberration → Chromatic Aberration, FringeBoost / Start / End / Center** — edge fringing and where it kicks in. The most likely values to turn distracting — tune them on the tablet at arm's length, not on a monitor. (*creates rgb fringing)

- **Scanlines → ScanlineCount / ScanlineDepth** - Tunes the look of the CRT scanlines - the number of scanlines (*240 worked well for the tablets resolution) and how distorted the scanline effect looks.
- **Jitter** — warps the screen in a sine wave in x and y like an old CRT monitor breaking down - keep this low or it looks crazy. The jitter can be fed the gyroscopes values in x and y from the gyro to also warp the whole image/tube slightly as the pitch and yaw of the device increases/decreases from the origin - this can make for interesting warping effects on screen which apply on-screen in the direction the player is tilting the gyroscope. (*be careful with tuning this one - crank it up and it is obvious)

6 · Player / allies — controls, movement, health

PlayerControl is a pivot at the tube's centre — rotating it swings the child ship around the wall, so steering is one angle rather than a position: gyro tilt sets the target angle, key input is added on top, and the ship eases toward it. Shooting spawns a projectile that sphere-casts down its own path, and every hit, heal and death runs through HealthComponent's callback arrays.

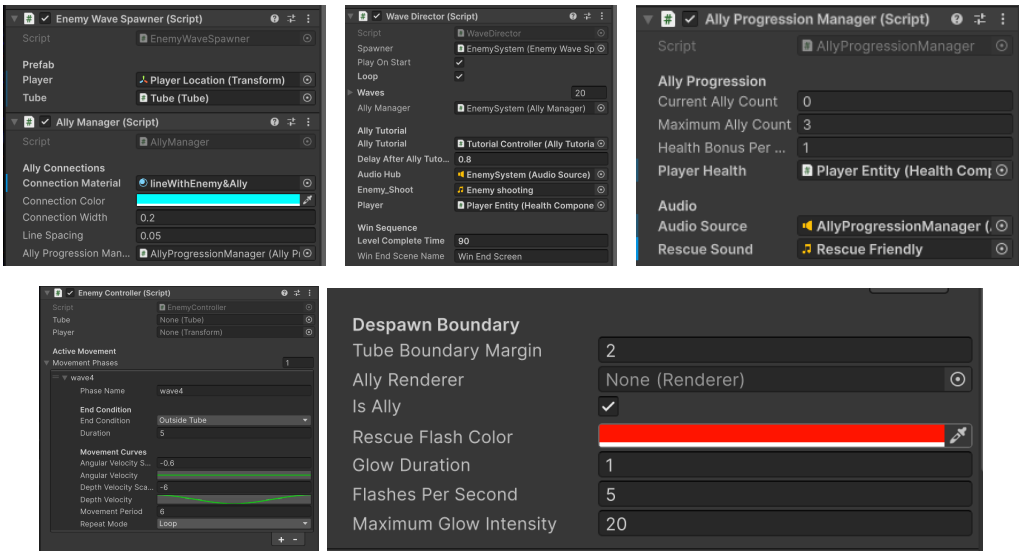


Inspector fields — Health Component · Player Shoot

- **Player control** — tiltRange is the device tilt needed for one full lap (at 90, a 45° tilt goes all the way round); moveRate is the key-turn speed; smoothing / homeSmoothing sets how hard the ship eases toward its target angle. (*tuned per device)
- **Player shoot** — Shoot Delay is the fire rate, Projectile Prefab and Spawnpoint set what fires and from where. On the projectile: moveSpeed, hitRadius (forgiveness on a near miss) and the enemyLayer mask it casts against.
- **Player health** — Health / Maximum Health plus the On Damage, On Health Changed and On Destroy arrays; drop any Callback in. PlayerDamageRecovery hooks in there with respawnDelay then invincibilityDuration.
- **Allies** — a rescued ally adds Health Bonus Per Ally up to Maximum Ally Count. Wave-side setup is below.

7 · Enemies logic

Enemies move in the tube's own terms — an angle around the wall and a depth (z) down it — both driven by AnimationCurves grouped into movement phases, so a path is authored in the Inspector rather than written in code, allowing designers to create orbiting, spiraling, stationary, inward, and outward movement without writing a new movement script. WaveDirector runs the waves on one timeline and does not manage individual enemy movement, EnemyWaveSpawner creates the members of a wave and assigns their initial tube coordinates, and one per wave can spawn as a rescuable ally. Enemy controller manages an individual enemy after it has spawned. Ally Manager tracks the ally and the enemies connected to it, It renders ally-to-enemy connection at runtime using line renderer components. We initially explored using random motion to have the enemies weave or spiral, but art-directing custom AnimationCurves ended up working better.



Enemy Wave Spawner & Ally Manager · Wave Director · Ally Progression Manager · Enemy Controller

- **Enemy types** — a type is an EnemyController prefab distinguished by its Movement Phases: each phase is an angularVelocity and a depthVelocity curve with scalers (angular is ×360, so 1 is a lap per period), a movementPeriod and a repeatMode.
- **End Condition** — Duration advances to the next phase, OutsideTube despawns past Tube Boundary Margin, UntilDestroyed waits for death. The last phase must be one of the latter two or the enemy sticks there and logs a warning.
- **Enemy waves** — each entry's launchTime is seconds from level start, so waves sit on one timeline rather than queueing. enemyCount with startAngle / angleSpacing and startDepth / depthSpacing shapes the formation, spawnInterval staggers the drop-in, and shootInterval / shootersPerVolley sets the volleys.
- **Allies** — allyIndex picks which member of the wave uses the Ally Prefab (negative for none). Ally Manager draws the connection lines; Ally Progression Manager holds Maximum Ally Count and the health bonus.